

Exercise #3

30. January 2023

Problem 1. (Rounding errors.)

It is known that, for smooth enough functions, the central finite difference approximation of second-order derivatives converges quadratically, that is,

$$\left| \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} - u''(x) \right| = O(h^2).$$

Computationally, however, this is only attainable for a certain range of h . If we keep reducing h , eventually the convergence will break down, since *machine accuracy* (or *machine precision* depending on the source) is usually limited to $O(10^{-16})$ values. Based on that, estimate the mesh size h (in order of magnitude) for which roundoff and approximation errors become comparable.

Solution.

Analogously to what seen in class for the first order case, the theoretical finite difference characteristic of the scheme we are considering is

$$\Delta u(x) = u(x-h) - 2u(x) + u(x+h).$$

Its rounded approximation is instead

$$\tilde{\Delta} u(x) = \Delta u(x) \pm \epsilon |u(x)|.$$

where $\epsilon > 0$. Taking into account the convergence property above, the error of the rounded finite difference formula can thus be rewritten as

$$\frac{\tilde{\Delta}u(x)}{h^2} - u''(x) = \overbrace{C(u^{(4)})h^2}^{\text{approximation error}} \pm \overbrace{\frac{\epsilon}{h^2}|u(x)|}^{\text{rounding error}}$$

where $C(u^{(4)})$ express the fact that the constant C depends on the fourth derivative of u . As long as the function u is regular enough, all the constants are of order one and can be ignored for the convergence. The errors will become comparable then when

$$|Ch^2| \approx |\epsilon/h^2| \implies h \approx (\epsilon/|C|)^{\frac{1}{4}}$$

The order of magnitude of h is thus $h = \mathcal{O}(\epsilon^{\frac{1}{4}})$. For a typical accuracy of $\epsilon = 10^{-16}$ we should then expect degradation when h reaches $(10^{-16})^{1/4} = 10^{-4}$.

Problem 2. (Rounding errors)

In this problem we are going to see how round off errors affect the solution to systems of linear equations.

Consider the problem of interpolating the polynomial $f(y) = 1 + y^2$ at the points $y = 0, 1, 2$, using a second degree polynomial. This can be done by solving the system of equations,

$$Ax = b, \quad A = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix},$$

where $x = (a_1, a_2, a_3)$ is the coefficients of the resulting polynomial (that is, our interpolating polynomial will look like $a_1 + ya_2 + y^2a_3$), and the vector b has entries $(f(0), f(1), f(2))$. Clearly, the solution to this system of equations is $(1, 0, 1)$.

When computing the solution x on a computer, the right hand side will not be exactly equal to b , but contain some error, $e \in \mathbb{R}^3$ (so in practice we are solving $Ax = b + e$).

The *condition number*, $\kappa(A)$, of the matrix A reflects how errors in the right hand side b propagate into the resulting solution x . It is defined as the maximum ratio of relative error in the right hand side and the in the solution,

$$\kappa(A) = \max_{b, e \neq 0} \frac{|A^{-1}e|}{|A^{-1}b|} / \frac{|e|}{|b|},$$

where $|\cdot|$ denotes the Euclidean norm. It turns out that $\kappa(A)^2$ is equal to the ratio of the largest and smallest eigenvalue of AA^T , so that $\kappa(A)^2 = \lambda_{\max}(AA^T)/\lambda_{\min}(AA^T)$.

- a) (J) Solve the system $Ax = b$ on your computer and compare the solution to $(1, 0, 1)$, using the relative error,

$$\frac{|A^{-1}e|}{|(1, 0, 1)|},$$

where $|\cdot|$ is the Euclidean norm.

Hint. The python function `np.linalg.norm(x, 2)` returns the Euclidean norm of a numpy array x .

- b) Compute the condition number of the matrix A (feel free to use any software to do so). Given that the relative error, $|e|/|b|$, in the right hand side is of order 10^{-10} , estimate the order of the relative error in the solution x .
- c) (J) Change the interpolation points to $0, 0.01, 2$, and make a new system of linear equations for the interpolating polynomial's coefficients. Solve this system on your computer and compare this with $(1, 0, 1)$. Finally compute the condition number of this system's matrix. Still using a relative error in b of order 10^{-10} , what is an upper bound for the order of the relative error in the solution now?
- d) Experiment numerically with different interpolation points in the interval $[0, 2]$, and see if you are able to get an error, $|(1, 0, 1) - A^{-1}b|$ of order 10^{-5} or bigger.

Solution.

- a) We get a small number of order 10^{-20} (this might vary from student to student, and depends the floating point number data type used, i.e. Python float, `np.float32` or `np.float64`).
- b) Computing the condition number we get 13.9. If the relative error in the right hand side is of order 10^{-10} , then the relative error in the solution will be of order $\kappa(A)10^{-10} \approx 10^{-9}$.

c) We get the new A matrix:

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 0.01 & 0.0001 \\ 1 & 2 & 4 \end{pmatrix}.$$

The relative error is computed to be $\approx 7 \cdot 10^{-16}$ (but this might vary). The condition number for this matrix is 730. This gives an upper bound for the relative error in the solution of order $730 \cdot 10^{-10} \approx 10^{-8}$.

d) If we change the interpolation point to $0, 10^{-7}, 10^{-6}$ we get a relative error in the solution of $1.3 \cdot 10^{-4}$, and an absolute error in the solution of $\sqrt{2} \cdot 1.3 \cdot 10^{-4} \approx 1.9 \cdot 10^{-4}$.

Problem 3. (Fixed-point method)

In this exercise, we will use fixed-point iterations to find the real root of

$$f(x) = 2x^3 - x^2 + 2x - 1 = 0. \quad (1)$$

To apply the fixed-point algorithm, we first need to rewrite Eq. (1) as $x = g(x)$, but this representation is not unique. For example, we can write

$$x = \frac{-2x^3 + x^2 + 1}{2} := g_1(x), \quad \text{or} \quad x = \sqrt[3]{\frac{x^2 - 2x + 1}{2}} := g_2(x).$$

In fact, the algorithm's convergence will depend on whether we choose g_1 or g_2 as our $g(x)$. We will experiment with both versions, and you can use the Jupyter notebook *05-Nonlinear-eqs.ipynb* for help.

- Find the expressions for $g'_1(x)$ and $g'_2(x)$ and, based on that, answer: would you rather select g_1 or g_2 as your $g(x)$? Why?
- (J) Set $x^{(0)} = 0.9$ and $g(x) = g_1(x)$, and iterate until $|x^{(k+1)} - x^{(k)}| < 10^{-6}$. What is the solution obtained, and how many iterations are needed to reach that?
- (J) Now, do the same for $g(x) = g_2(x)$. How many iterations are now needed?

Solution.

a) We have:

$$g'_1(x) = \frac{-6x^2 + 2x}{2} = -3x^2 + x, \quad (2)$$

$$g'_2(x) = \frac{\partial}{\partial x} \left[\frac{(x-1)^{2/3}}{2^{1/3}} \right] = 2^{-1/3} \frac{2}{3} (x-1)^{-1/3} = \frac{2^{2/3}}{3} (x-1)^{-1/3} = \frac{2^{2/3}}{3} \frac{1}{(x-1)^{1/3}}, \quad (3)$$

Once close enough to the solution, the size of the derivative $g'_i(x)$ determines the convergence speed. The one that is closer to 0 is generally preferable and for this $g_1(x)$ is to be preferred.

Problem 4. (Fixed-point iterations)

In pipeline design for oil transport, pressure losses must be carefully estimated. They are directly proportional to a positive friction factor f_D , whose inverse square root $x := 1/\sqrt{f_D}$ is given by a non-linear equation. In fact, it is known that x depends only on the Reynolds number R (look up “*Reynolds number*” online, if you are interested in fluid mechanics). For a turbulent flow at a given R , the equation to find x is

$$x + 1.93 \ln(1.32x/R) = 0,$$

in which $\ln(x)$ denotes the natural logarithm. In this exercise, we will set $R = 5000$ and use fixed-point iterations to approximate x , according to

$$x^{(k+1)} = g(x^{(k)}), \quad \text{with } g(x) := -1.93 \ln(1.32x/5000). \quad (4)$$

Let us also define the iteration error as $e_k := |x^{(k)} - x^{(k-1)}|$. Based on that, you are asked to:

- Compute $g'(x)$ and use the result to determine whether $g(x)$, $x > 0$, is an increasing, decreasing or a non-monotonic function.
- Calculate the maximum and minimum values of $g(x)$ in the interval $x \in [e, e^3]$.
- Determine the maximum value L of $|g'(x)|$ in the interval $[e, e^3]$.
- For $x^{(0)} = e^2$, perform the first fixed-point iteration for the solution of Eq. (4).
- Based on the values of L , $x^{(0)}$ and $x^{(1)}$, find an upper bound for the number of iterations required to reach a tolerance of 10^{-3} .

Solution.

a) We have

$$g'(x) = \left[-1.93 \ln \left(\frac{1.32}{5000} x \right) \right] \Big|_x = -1.93 \frac{1}{\frac{1.32}{5000} x} \frac{1.32}{5000} = -\frac{1.93}{x}$$

For $x > 0$, $g'(x)$ is negative thus $g(x)$ is a decreasing function there.

b) Knowing that $g'(x)$ is monotone, its maximum and minimum are attained at the boundary of the interval. We have:

$$\begin{aligned} g'(e) &\approx -0.71 \\ g'(e^3) &\approx -9.6e^{-2} \end{aligned}$$

We can conclude that for $|g'(x)|$:

$$\max_{[e, e^3]} |g'(x)| = |g'(e)| \approx 0.71$$

c) The first fixed point iteration is:

$$x^{(1)} = -1.93 \ln(1.32x^{(0)}/5000) = -1.93 \ln(1.32e^2/5000) \approx 12.04$$

d) We now have all ingredients to follow the Banach's Fixed Point Theorem, that states: given an interval where g is continuously differentiable and where $|g'(x)|$ can be bounded by a constant $L < 1$, the error at a certain iteration n can be bounded by

$$|x - x^{(n)}| \leq \frac{L^n}{1 - L} |x^{(1)} - x^{(0)}|.$$

We found, in point c) the values for $x^{(1)}$, $x^{(0)}$ and we can take, given point b), $L = 0.71$.

e) Suppose we want to reach a solution for which $|x - x^{(n)}| < 10^{-3}$, so that our error is low enough. This, looking at point d), can be guaranteed if we can reach

$$\frac{L^n}{1 - L} |x^{(1)} - x^{(0)}| < 10^{-3}.$$

At this point, the only unknown is n .

f) In order to find n we can take the logarithm on both sides, obtaining

$$\ln(L^n) + \ln\left(\frac{|x^{(1)} - x^{(0)}|}{1 - L}\right) < \ln(10^{-3}) \Rightarrow n \ln(L) + \ln\left(\frac{|x^{(1)} - x^{(0)}|}{1 - L}\right) < -3 \ln(10)$$

Rearranging we have:

$$n > \frac{1}{\ln(L)} \left[-3 \ln(10) - \ln\left(\frac{|x^{(1)} - x^{(0)}|}{1 - L}\right) \right]$$

where the inequality sign is reversed since we have to divide by $\ln(L)$ which is negative. Using previously defined quantities we have $|x^{(1)} - x^{(0)}| \approx 12.042 - e^2 \approx 4.653$ and $L \approx 0.71$. Plugging the actual numbers we obtain

$$n > 29.$$

This means that in the worst case after $n = 29$ iterations our solution will have the precision needed. Since this is the worst case scenario, $n = 29$ constitutes our upper bound.